

Experiment 2 : Merge Sort

Code :-

```
import multiprocessing
import time

# Merge function
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Sequential merge sort
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
```

```

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
# Parallel merge sort
def parallel_merge_sort(arr):
    if len(arr) <= 1000:
        return merge_sort(arr)
    mid = len(arr) // 2
    with multiprocessing.Pool(2) as pool:
        left, right = pool.map(merge_sort, [arr[:mid], arr[mid:]])
    return merge(left, right)
# Main
if __name__ == "__main__":
    n = int(input("Enter number of elements: "))
    arr = list(map(int, input("Enter elements: ").split()))
    start = time.time()
    sorted_seq = merge_sort(arr)
    print("Sequential Time:", time.time() - start)
    start = time.time()
    sorted_par = parallel_merge_sort(arr)
    print("Parallel Time:", time.time() - start)
    print("Sorted Array:", sorted_par)

```

output :-

Enter number of elements: 8

Enter elements: 8 3 1 7 0 10 2 5

Sequential Time: 2.4557113647460938e-05

Parallel Time: 2.288818359375e-05

Sorted Array: [0, 1, 2, 3, 5, 7, 8, 10]